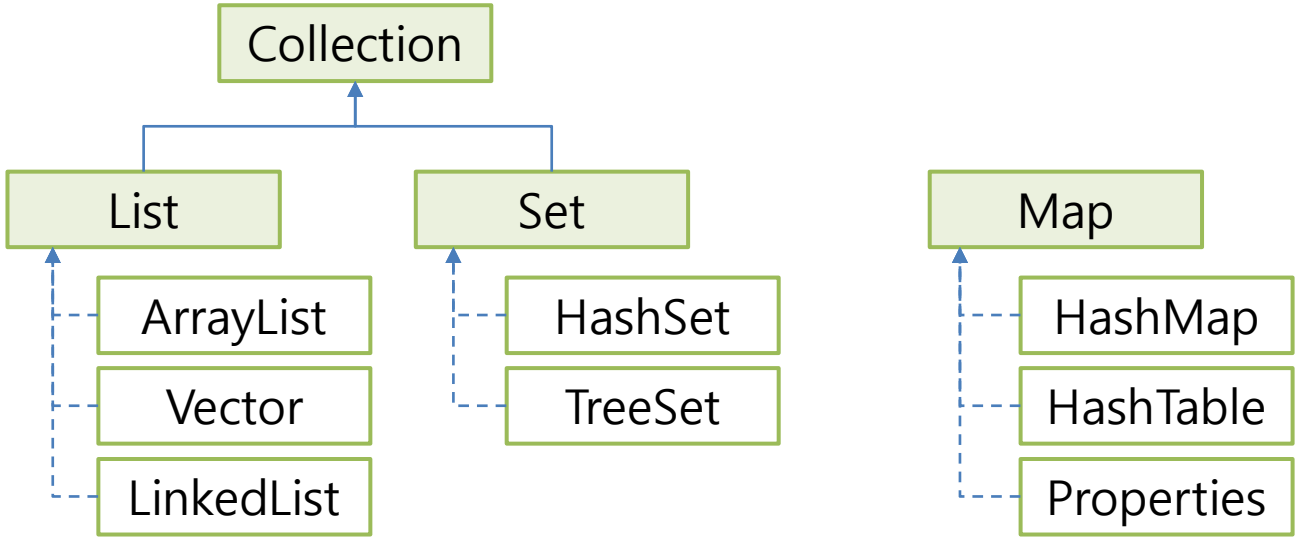
20강. JAVA Collections

- 컬렉션 프레임워크
- List 컬렉션
- Set 컬렉션
- Map 컬렉션

20-1. 컬렉션 프레임워크

- 자바의 널리 알려져 있는 자료구조(Data Structure)를 기준으로 객체를 효율적으로 관리할 수 있도록 관련 인터페이스와 클래스들을 java.util 패키지에 포함시켜 놓았습니다. 이것들을 총칭해서 컬렉션 프레임워크(Collection Framework)라고 한다.
- 주요 인터페이스로는 List, Set, Map 이 있고, 이 인터페이스로 사용 가능한 컬렉션 객체는 아래와 같다.



- List와 Set은 객체 처리방법에 있어 공통점이 있어 공통된 메서드만 따로 모아 Collection 인터페이스를 정의해 두고 이것을 상속한다.
- Map은 키와 값을 하나의 쌍으로 관리하는 구조, List 및 Set과 사용방법이 다르다.

20-1. 컬렉션 프레임워크

➢ 컬렉션의 특징 별 분류

인터페이스 분류		특징	구현 클래스
Collection	List	- 순서를 유지하고 저장 - 중복 저장 가능	ArrayList, Vector LinkedList
	Set	- 순서를 유지하지 않고 저장 - 중복 저장 안됨	HashSet, TreeSet
Map		- 키와 값으로 구성된 엔트리 저장 - 키는 중복 저장 안됨	HashMap, HashTable Properties

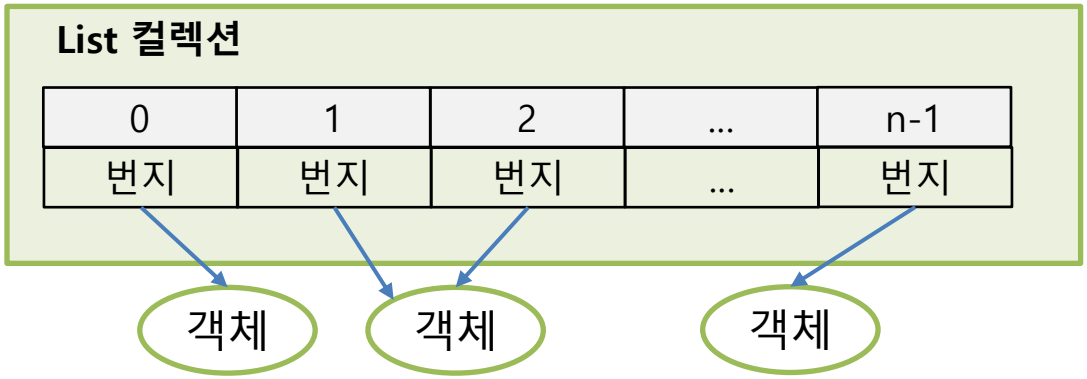
20-2. List 컬렉션

- 객체를 인덱스로 관리한다. 객체를 저장하면 인덱스가 부여되고 인덱스로 객체를 검색, 삭제할 수 있는 기능을 제공한다.
- List 컬렉션 : ArrayList, Vector, LinkedList
- List 컬렉션에서 공통적으로 사용 가능한 List 인터페이스 메서드

기능	메서드	설명
객체 추가	boolean add(E e)	주어진 객체를 맨 끝에 추가
	void add(int index, E element)	주어진 인덱스에 객체 추가
	set(int index, E element)	주어진 인덱스의 객체를 새로운 객체로 교체
객체 검색	boolean contains(Object o)	주어진 객체가 저장되어 있는지 여부
	E get(int index)	주어진 인덱스에 저장된 객체를 리턴
	isEmpty()	컬렉션이 비어 있는지 조사
	int size()	저장되어 있는 전체 객체 수 리턴
객체 삭제	void clear()	저장된 모든 객체를 삭제
	E remove(int index)	주어진 인덱스에 저장된 객체를 삭제
	boolean remove(Object o)	주어진 객체를 삭제

20-2. List 컬렉션 - ArrayList

- List 컬렉션에서 가장 많이 사용하는 컬렉션이다.
- ArrayList에 객체를 추가하면 내부 배열에 객체가 저장되며, 일반 배열과 달리 제한 없이 객체를 추가할 수 있다.



- List 컬렉션은 객체의 번지를 저장한다.
- 동일한 객체를 중복 저장 할 수 있고, 이 경우 동일한 번지가 저장된다.
- null 도 저장 가능하다.

```
List<E> list = new ArrayList<E>();
```
- E에 지정된 타입의 객체만 저장

```
List<E> list = new ArrayList<>();
```
- E에 지정된 타입의 객체만 저장

```
List list = new ArrayList<>();
```
- 모든 타입의 객체를 저장

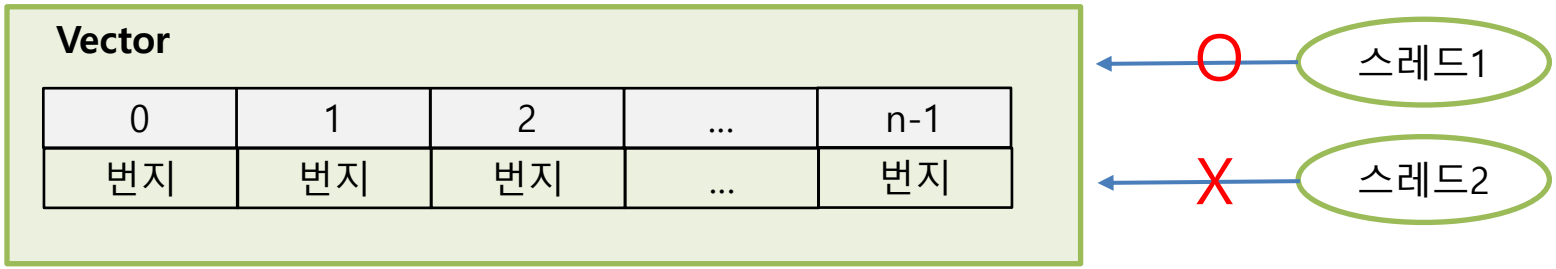
20-2. List 컬렉션 - ArrayList

➢ 사용 예

```
public class Board {
    private String title;
    private String contents;
    private String writer;

    Board(String title, String contents, String writer) {
        this.title = title;
        this.contents = contents;
        this.writer = writer;
    }

    @Override
    public String toString() {
        String strBoard = "제목: " + title + ", 내용: " + contents + ", 작성자: " + writer;
        return strBoard;
    }
}
  public static void main(String[] args) {
        List<Board> list = new ArrayList<Board>();

        Board board1 = new Board("제목1", "내용1", "작성자1");
        Board board2 = new Board("제목2", "내용2", "작성자2");

        list.add(board1);
        list.add(board2);

        System.out.println("게시글: " + list.get(1));
  }
```

20-2. List 컬렉션 – Vector

- ArrayList와 동일한 내부 구조를 가지고 있다.
- Vector는 동기화(synchronized)된 메서드로 구성되어 있어 멀티 스레드가 동시에 Vector() 메서드를 실행할 수 없다. 그래서 멀티 스레드 환경에서 안전하게 객체 처리(추가/삭제)가 가능하다.



- List 컬렉션은 객체의 번지를 저장한다.
- 동일한 객체를 중복 저장 할 수 있고, 이 경우 동일한 번지가 저장된다.
- null 도 저장 가능하다.

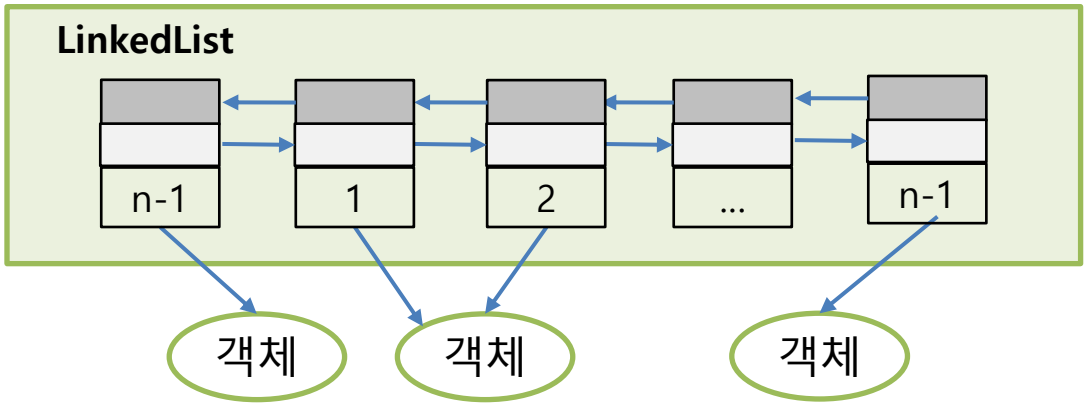
```
List<E> list = new Vector<E>();
```
- E에 지정된 타입의 객체만 저장

```
List<E> list = new Vector<>();
```
- E에 지정된 타입의 객체만 저장

```
List list = new Vector();
```
- 모든 타입의 객체를 저장

20-2. List 컬렉션 – LinkedList

- ArrayList와 완전히 다른 내부 구조를 가지고 있다.
- LinkedList는 인접 객체를 체인처럼 연결해서 관리한다.



- LinkedList는 특정 위치에 객체를 삽입하거나 삭제 시 바로 앞뒤 링크만 변경하면 된다.
- 빈번한 삭제와 삭제 시 ArrayList 보다 더 효율적이다.

```
List<E> list = new LinkedList<E>();
List<E> list = new LinkedList<>();
List list = new LinkedList();
```

- E에 지정된 타입의 객체만 저장
- E에 지정된 타입의 객체만 저장
- 모든 타입의 객체를 저장

20-3. Set 컬렉션

- 중복 객체를 허용하지 않고 저장 순서가 없다. 수학의 집합 개념과 유사
- Set 컬렉션 : HashSet, LinkedHashSet, TreeSet

- Set 컬렉션에서 공통적으로 사용 가능한 Set 인터페이스 메서드

기능	메서드	설명
객체 추가	boolean add(E e)	주어진 객체를 저장 성공하면 true, 중복 객체면 false 리턴
객체 검색	boolean contains(Object o)	주어진 객체가 저장되어 있는지 여부
	Iterator<E> iterator()	저장된 객체를 한번씩 가져오는 반복자 리턴
	isEmpty()	컬렉션이 비어 있는지 조사
	int size()	저장되어 있는 전체 객체 수 리턴
객체 삭제	void clear()	저장된 모든 객체를 삭제
	boolean remove(Object o)	주어진 객체를 삭제

20-2. Set 컬렉션 - HashSet

- Set 컬렉션에서 가장 많이 사용하는 컬렉션이다.
- 동일한 객체를 중복 저장하지 않는다. 순서도 없다.

```
Set<E> set = new HashSet<E>();
```
- E에 지정된 타입의 객체만 저장

```
Set<E> set = new HashSet<>();
```
- E에 지정된 타입의 객체만 저장

```
Set set = new HashSet();
```
- 모든 타입의 객체를 저장

```
Set<String> set = new HashSet<String>();

set.add("apple");
set.add("banana");
set.add("orange");
set.add("apple");    // 중복 객체 저장하지 않음

System.out.println("Set size: " + set.size());

// Set 순회
for(String fruit: set) {
    System.out.println(fruit);
}
```

20-4. Map 컬렉션

- Map 컬렉션은 키(key)와 값(value) 쌍으로 구성된 엔트리(entry) 객체를 저장한다.
- 키와 값은 모두 객체이고, 키는 중복 저장할 수 없으나, 값은 중복 저장 가능하다.
- Map 컬렉션 : HashMap, HashTable, LinkedHashMap, Properties, TreeMap
- Map 컬렉션에서 공통적으로 사용 가능한 Map 인터페이스 메서드

기능	메서드	설명
객체 추가	put(K key, V value)	주어진 키와 값을 추가, 저장이 되면 값을 리턴
객체 검색	containsKey(Object key)	주어진 키가 있는 지 여부
	containsValue(Object value)	주어진 값이 있는지 여부
	get(Object key)	주어진 키의 값을 리턴
	Iterator<E> iterator()	저장된 객체를 한번씩 가져오는 반복자 리턴
	boolean isEmpty()	컬렉션이 비어 있는지 조사
	Set<K> keyset()	모든 키를 Set 객체에 담아 리턴
	int size()	저장되어 있는 전체 객체 수 리턴
객체 삭제	void clear()	저장된 모든 객체를 삭제
	remove(Object key)	주어진 키와 일치하는 Map.Entry 삭제, 삭제되면 값을 리턴

20-4. Map 컬렉션 – HashMap

➢ 해시 테이블을 사용하여 키-값 쌍을 저장하고, 해시 테이블은 빠른 검색, 삽입, 삭제를 제공한다.

```
Map<K, V> map = new HashMap<K, V>();
```

키 타입 값 타입 키 타입 값 타입

```
Map<String, Integer> map = new HashMap<String, Integer>();

map.put("김길동", 85);
map.put("홍길동", 95);          // 키가 같아 마지막에 저장한 값만 저장
map.put("길홍이", 95);
map.put("홍길동", 87);          // 키가 같아 마지막에 저장한 값만 저장
System.out.println("총 Entry: " + map.size());

int value = map.get("홍길동");      // 키를 매개값으로 주면 값을 리턴
System.out.println("value: " + value);
```

➢ 요점 정리

컬렉션 프레임워크

- 자바의 자료 구조를 사용하여 객체를 효율적으로 관리할 수 있도록 클래스와 인터페이스를 만들어 놓은 것을 컬렉션 프레임워크라고 한다.

List 컬렉션

- List 컬렉션에서 가장 많이 사용되는 것은 ArrayList 컬렉션이다.
- 순서가 있고 중복 저장이 가능하다.
- 객체를 인덱스를 사용하여 내부 배열로 관리한다. 용량 제한이 없다.

Map 컬렉션

- Map 컬렉션에서 가장 많이 사용되는 것은 HashMap 컬렉션이다.
- 키와 값은 모두 객체이고, 키는 중보 저장할 수 없으나 값은 중복 저장 가능하다.
- 키-값 쌍을 저장하는 해시 테이블 기반의 자료 구조로, 많은 양의 데이터를 빠르게 검색하고 접근할 수 있다.

➢ 예제 1

다음을 수행하는 코드를 작성하세요.

① 이름을 가지는 Person 클래스를 작성하세요. Person은 반드시 이름을 포함해야 합니다.
② 이름이 "홍길동", "만득이" 인 Person 인스턴스를 생성하고, ArrayList에 담습니다.
③ ArrayList에 담긴 모든 person 인스턴스의 이름을 표시하세요.

➢ 예제 2

BoardDao 객체의 getBoardList() 메서드를 호출하면 List<Board> 타입의 컬렉션을 리턴합니다. ArrayListEx 클래스의 실행 결과를 보고, BoardDao 클래스와 getBoardList() 메서드를 작성하세요.

```java
public class Board {
    private String title;
    private String contents;

    Board(String title, String contents) {
        this.title = title;
        this.contents = contents;
    }

    public String getTitle() { return title; }
    public String getContents() { return contents; }

    @Override
    public String toString() {
        String strBoard = "제목: " + title + ", 내용: " + contents;
        return strBoard;
    }
}
```

```java
public class ArrayListEx {
    public static void main(String[] args) {
        BoardDao dao = new BoardDao();
        List<Board> list = dao.getBoardList();

        for(Board board:list) {
            System.out.println(board.toString());
        }
    }
}
```